PROJECT REPORT

Project:
Multi-Tenant Course Selling App

Course:
Database Management System (UCS310)

Department:
COE (2C82)

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Deemed to be University), Patiala, Punjab

Submitted By:

Name	Roll Number
Ishaan Gupta	(1024031118)
Saksham Kansal	(1024031135)
Sanshi	(1024031137)

Submitted To:
Mrs. Damini Arora

Academic Year: 2025-2026

1. Introduction

The proposed project is a multi-tenant SaaS-based course selling platform that enables instructors to create and manage their own courses while allowing students to enroll in them. The online education domain was chosen due to its need for structured data management, secure user handling, and reliable transactions.

A Database Management System (DBMS) is essential for this system as it manages multiple related entities such as instructors, students, courses, enrollments, and content. Compared to file-based systems, a DBMS ensures data consistency, reduces redundancy, supports efficient querying, and enables secure concurrent access, making it suitable for scalable applications.

2. Problem Statement

Independent educators often use fragmented or manual systems to manage courses, students, and enrollments, leading to data duplication, poor access control, and limited scalability.

This project addresses the need for a centralized, database-driven system that efficiently handles instructor management, student authentication, course management, content organization, and enrollments. A relational DBMS ensures structured relationships, data integrity, and consistent operations across the platform.

3. Objectives of the Project

- To design a relational database using E–R data modeling techniques
- To convert the ER model into a well-defined relational schema
- To apply normalization up to Third Normal Form (3NF)
- To implement the database using SQL with proper constraints
- To ensure data integrity using primary keys, foreign keys, and unique constraints
- To support backend CRUD and relational data retrieval operations

4. Scope of the Project

- Instructor registration and authentication
- Student registration under instructor-specific contexts
- Course creation, modification, and listing
- Course content organization using folders and content entities
- Enrollment management linked with payment confirmation

The system involves three primary user roles: **Instructor**, **Student**, and **Admin**. The project focuses mainly on backend database design and implementation, with APIs consuming and managing structured data.

5. Proposed System Description

The proposed system is a SaaS-based backend-driven application where instructors register on the main platform and receive a unique identifier. Students interact within the context of an instructor's domain. The backend manages authentication, authorization, course data, enrollments, and content metadata.

The relational database ensures strong data integrity through enforced constraints, improves efficiency through optimized queries, and enhances consistency by maintaining well-defined relationships among users, courses, and enrollments.

6. Database Design

6.1 Entity–Relationship (ER) Diagram:

Entities:

- Admin
- Instructor
- Student
- Course
- CourseFolder
- CourseContent
- Enrollment
- Payment

Attributes:

- **Instructor:** id, name, email, organization, slug
- **Student:** id, name, email, instructorId
- **Course:** id, title, price, level, type
- **Payment:** amount, status, payment IDs

Relationships:

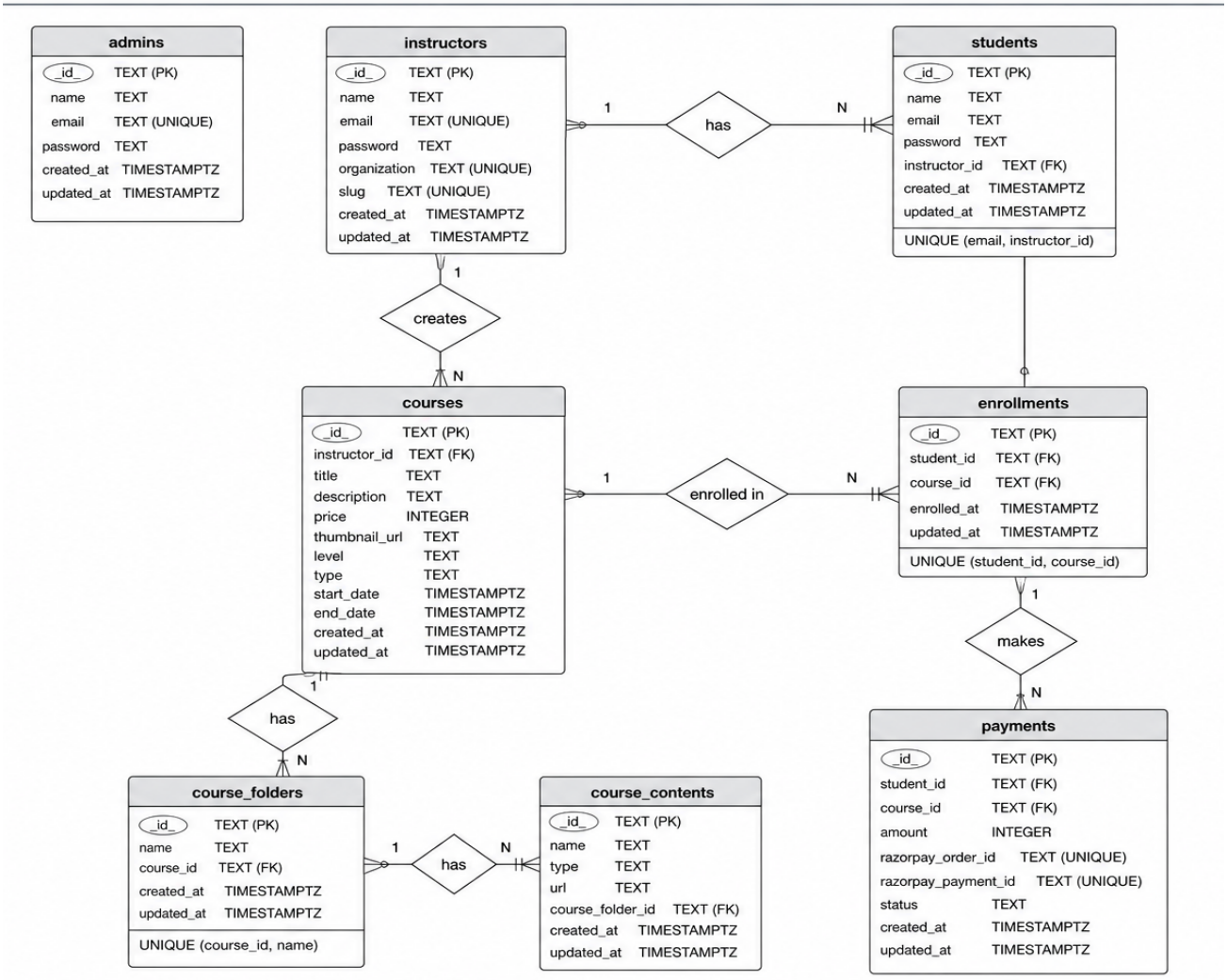
- **Instructor → Course**
  One instructor can create multiple courses.
- **Instructor → Student**
  One instructor can have multiple students.
- **Course → CourseFolder**
  A course can contain multiple folders.
- **CourseFolder → CourseContent**
  Each folder can contain multiple content items.

- **Student ↔ Course (via Enrollment)**
  Many-to-many relationship where students can enroll in multiple courses and each course can have many students.
- **Student → Payment**
  A student can make multiple payments.
- **Course → Payment**
  Each course can have multiple associated payments.

Cardinality & Constraints:

- **One-to-Many (1:N):**
  - Instructor → Course
  - Instructor → Student
  - Course → CourseFolder
  - CourseFolder → CourseContent
- **Many-to-Many (M:N):**
  - Student ↔ Course (resolved using Enrollment table)

ER Diagram:



6.2 Relational Schema

- **Admin** (id PK, name, email UNIQUE, password, created_at, updated_at)
- **Instructor** (id PK, name, email UNIQUE, password, organization UNIQUE, slug UNIQUE, created_at, updated_at)
- **Student** (id PK, name, email, password, instructorId FK, created_at, updated_at, UNIQUE(email, instructorId))
- **Course** (id PK, instructorId FK, title, description, price, thumbnail_url, level, type, start_date, end_date, created_at, updated_at)
- **CourseFolder** (id PK, name, courseId FK, created_at, updated_at, UNIQUE(courseId, name))
- **CourseContent** (id PK, name, type, url, courseFolderId FK, created_at, updated_at)
- **Enrollment** (id PK, studentId FK, courseId FK, enrolled_at, updated_at, UNIQUE(studentId, courseId))
- **Payment** (id PK, studentId FK, courseId FK, amount, razorpay_order_id UNIQUE, razorpay_payment_id UNIQUE, status, created_at, updated_at)

7. Normalization

- **1NF:** All attributes contain atomic values
- **2NF:** Partial dependencies are eliminated using composite uniqueness where required
- **3NF:** Non-key attributes depend only on the primary key, with transitive dependencies removed through separate entities

8. Database Implementation

8.1 SQL Implementation:

DDL commands:

- **CREATE:** (instructors, students, courses, admin) table
- **DROP Cascade:** (instructors, students, courses, admin)

DML commands:

- **Insert:** (instructors, students, courses, admin) data.
- **Update:** (instructors, students, courses, admin) data.
- **Delete:** (instructors, students, courses, admin) data.

SELECT:

- **JOINS:** Courses with Instructor Names
- **Aggregate Functions:** Total Revenue Calculation
- **Subqueries:** Courses Above Average Price
- **Group By & Having:** Enrollments per Course
- **Views:** Platform Stats retrieving all students, instructors, courses, total revenue

8.2 PL/SQL Components:

Trigger: This trigger is executed **before every UPDATE operation** on the courses table. It automatically updates the updated_at column with the current timestamp.

- trg_courses_updated_at

Function: A function that returns the total number of students enrolled in a specific course.

- get_total_enrollments(course_id TEXT)

Stored Procedure: A procedure to safely enroll a student into a course while preventing duplicate enrollments.

- enroll_student

Cursor: A cursor used to iterate through courses and display enrollment counts.

- Course-wise Enrollment Cursor

Exception Handling: Handles errors during payment insertion to prevent system failure.

- Safe Payment Insert

9. Transaction Management & Concurrency

Use of Transactions:

- The transaction begins with BEGIN and ends with COMMIT, ensuring all operations execute as a single unit.
- Payment is updated only if it is in PENDING state, and enrollment is inserted using ON CONFLICT to prevent duplicate entries; failures can be rolled back.

Concurrency Control:

- Conditional updates help prevent race conditions during concurrent operations.
- ON CONFLICT and row-level locking ensure safe concurrent inserts and maintain data consistency.

10. Tools & Technologies Used

- **Database:** PostgreSQL (pgAdmin)
- **Backend Framework:** Node.js with Express.js
- **Frontend Framework:** React.js
- **Integrations:** AWS S3, Razorpay

11. Current Outcomes:

The current outcome of this project is a fully normalized, relational database full-stack SaaS course platform that supports secure role-based workflows, efficient data retrieval, structured content management, and reliable enrollment handling. The system ensures improved data consistency, integrity, and scalability.

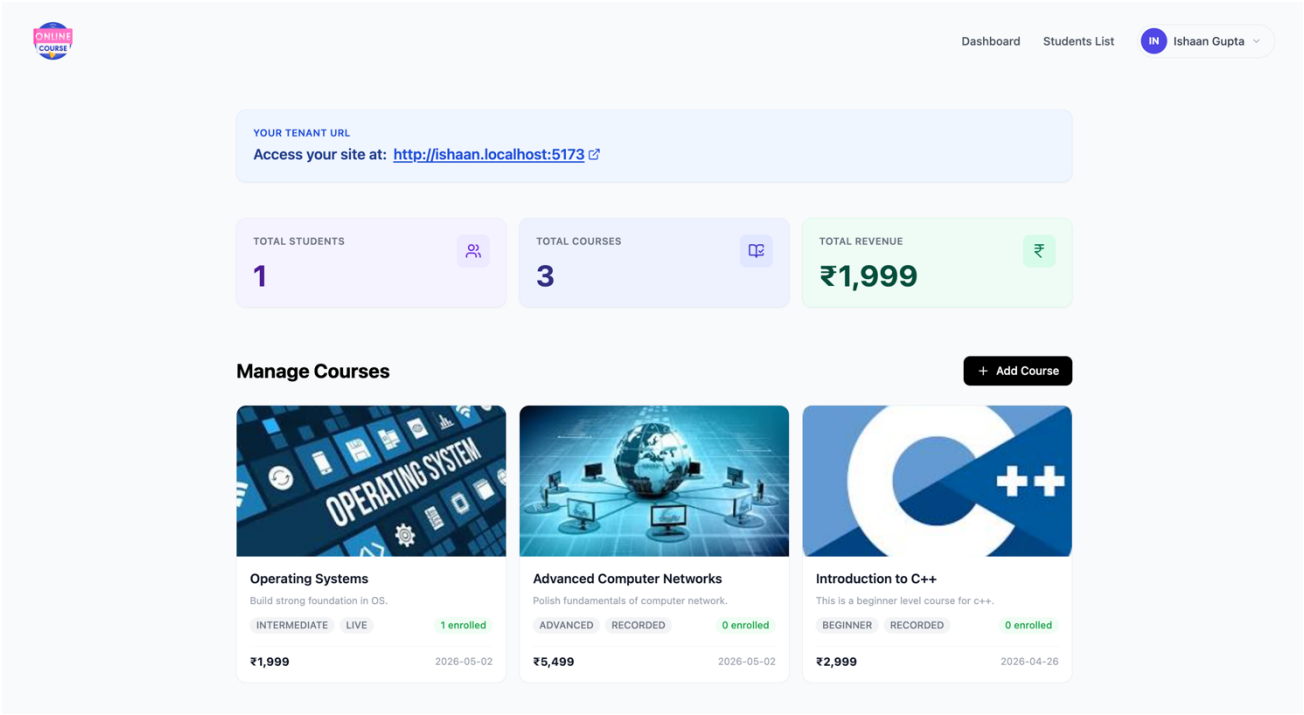


Figure 1

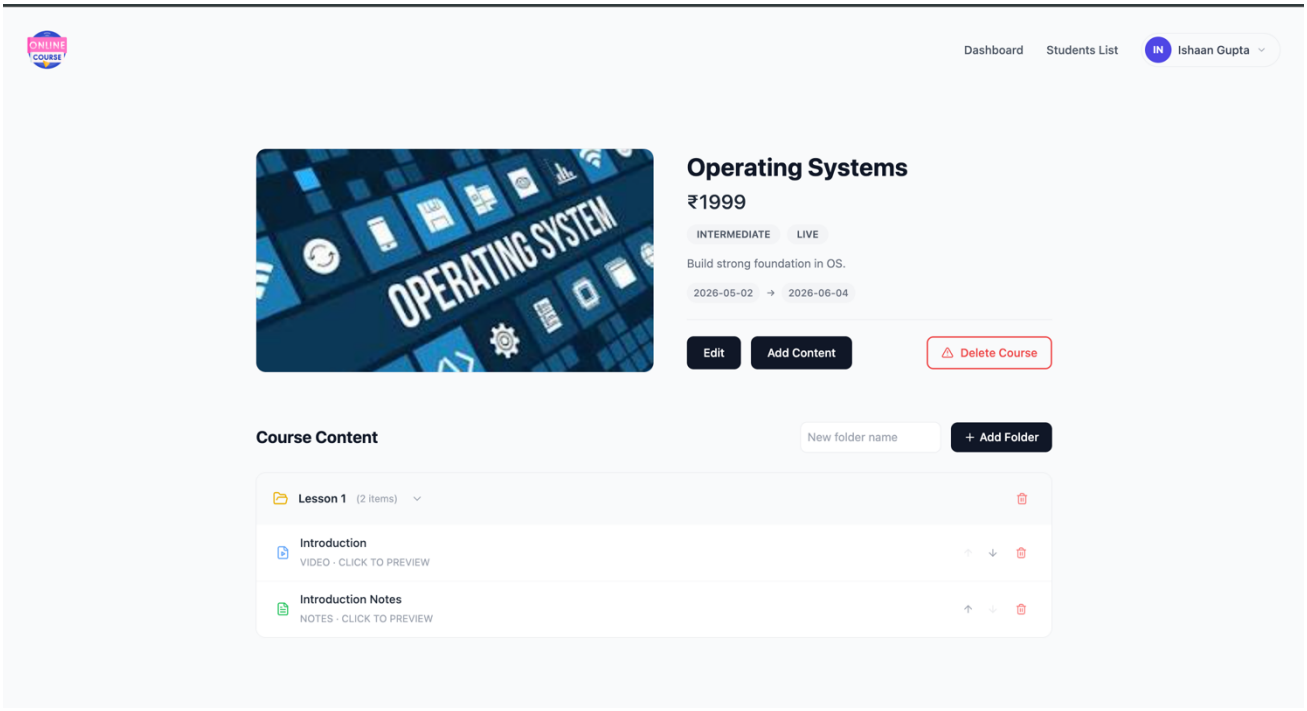


Figure 2